Algoritmo

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos de R

R es un lenguaje muy parecido a mathlab y permite el trabajo con matrices.

```
a=38
a=40
b=80
c=b-a
c
```

```
## [1] 40
```

```
y=40
alpha<-30
```

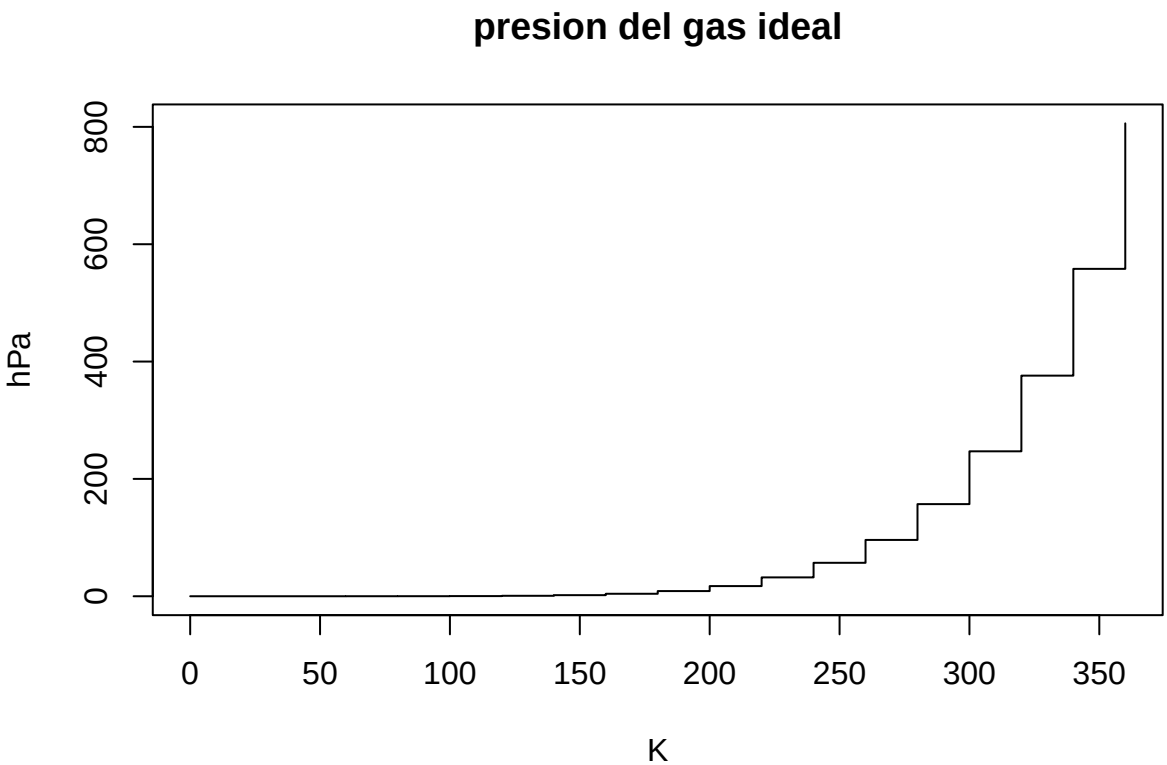
##uso de datasets o base de datos internos de R usando el comando data en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan dados en tablas de las cuales podemos elegir columnas especificas si escribimos $ depues del comando de informacion especifico y elegimos la columa de datos deseada

```
summary(cars)
```

```
##      speed           dist       
##  Min.   : 4.0   Min.   :  2.00  
##  1st Qu.:12.0   1st Qu.: 26.00  
##  Median :15.0   Median : 36.00  
##  Mean   :15.4   Mean   : 42.98  
##  3rd Qu.:19.0   3rd Qu.: 56.00  
##  Max.   :25.0   Max.   :120.00  
```

Including Plots

You can also embed plots, for example:



##comando plot El comando plot es un comando que permite graficar cualquier fuente de datos que le asignemos, teniendo tambien la posibilidad de asignar nombre a las variables o hacer cambios al grafico. En caso de duda de como utilizar un comando podemos escribir ##plot## en la consola, lo cual abrira una ventana que explicara en detalle el comando.

##Funciones estadisticas

```
z1<-rnorm(350,22,5)
z1
```

```
##   [1] 22.91888 17.74777 17.92503 13.90381 20.30961 27.65921 29.11869 24.61806
##   [9] 26.00493 26.17247 18.97145 25.36225 27.05143 20.30117 16.90232 31.08131
##  [17] 23.00921 17.23043 29.37263 18.46687 30.71221 26.46027 25.77739 24.51848
##  [25] 17.61133 23.76518 17.94641 21.43095 25.88321 23.50687 23.96640 25.36083
##  [33] 26.47320 30.88241 31.44288 14.46265 25.40713 16.94052 21.52904 20.71129
##  [41] 23.37754 18.22843 24.02257 22.42464 20.62640 20.80967 23.49173 30.56361
##  [49] 28.77414 24.06295 14.81236 24.78428 22.17636 23.71954 27.03333 19.19921
##  [57] 24.05720 22.11423 16.12748 20.78174 24.65012 27.98613 31.29321 16.97783
##  [65] 17.55028 31.13439 21.88469 13.30318 19.61977 24.16085 27.38469 24.12021
##  [73] 24.66457 13.93035 19.17380 33.40313 19.58660 22.13823 25.64347 13.70900
##  [81] 26.23040 21.28289 11.83663 16.47307 26.73690 18.17716 22.08637 23.73979
##  [89] 21.78148 22.38242 19.04561 22.17917 17.84666 23.94040 19.49555 19.18234
##  [97] 19.88586 19.25799 26.38053 22.63280 23.53282 23.20249 32.53034 19.05473
## [105] 19.17639 14.88701 18.56139 16.23723 28.14568 24.28409 13.70420 36.69986
## [113] 19.90879 22.35902 22.78245 22.15433 15.11460 26.85716 30.79622 16.97751
## [121] 18.58032 27.65790 23.49820 28.96891 17.44687 23.69466 28.61207 16.89672
## [129] 25.94413 25.61043 19.66651 26.20560 24.63701 25.39248 20.26427 21.85508
## [137] 21.17053 21.51941 16.55443 24.98349 23.48941 25.49770 24.11741 14.56335
## [145] 22.28962 25.80335 23.50865 18.95980 24.98488 31.94850 18.77609 12.06332
## [153] 18.01074 26.83878 21.90700 20.16980 20.78028 18.14341 19.49957 29.09473
## [161] 19.04608 12.84631 22.38916 12.67106 22.78720 14.69331 15.94954 24.37418
```

```
## [169] 12.87857 18.81169 19.21554 35.05058 18.09183 22.70136 22.74974 19.26856
## [177] 23.85466 22.03538 20.55062 21.54969 20.90084 19.99043 17.85084 14.99241
## [185] 16.16571 16.76581 26.37848 17.50370 15.64534 27.78551 21.03153 20.33356
## [193] 16.39854 21.59188 24.56867 21.47830 22.61683 22.19077 15.37865 28.67180
## [201] 23.87324 20.27728 19.56441 30.91934 21.88729 26.56970 21.08238 17.44675
## [209] 22.28239 24.69940 27.71917 16.19568 21.14419 24.99260 18.53588 24.49079
## [217] 20.70933 26.27354 17.01070 14.79971 22.99524 28.23196 12.87759 18.28841
## [225] 15.87508 22.95542 21.20979 22.31942 19.68681 22.99036 24.48432 23.67440
## [233] 21.72030 24.09547 27.69371 15.49071 13.12645 18.45355 16.25752 31.46560
## [241] 17.58106 24.59141 18.56223 18.28546 19.18592 14.56907 25.27200 16.24902
## [249] 24.17369 29.47133 19.28934 23.31467 24.33035 34.72791 18.09261 28.12664
## [257] 16.50002 23.13637 19.81090 24.53623 21.76651 25.30500 19.87560 26.30017
## [265] 19.62796 26.62338 25.07146 23.55118 16.54903 19.54470 18.24701 21.05402
## [273] 28.47490 27.36265 19.25777 24.19644 19.41427 28.97198 30.18089 17.41449
## [281] 22.85268 23.13563 26.79407 17.92228 28.44461 26.65003 24.25626 23.17661
## [289] 20.20312 23.97277 16.70547 25.16014 32.58506 18.36357 21.77733 22.00692
## [297] 27.98553 26.07136 21.41652 18.46735 23.21396 22.51488 26.44894 23.34630
## [305] 23.18967 22.11327 18.13941 11.45727 24.64655 20.09200 23.21294 20.96656
## [313] 26.95804 22.62655 20.56992 24.57343 25.54382 28.36092 24.29170 18.42432
## [321] 23.31113 17.85880 18.16805 13.76730 27.10770 31.49324 19.67412 33.17412
## [329] 19.31972 23.48076 21.24912 31.20497 23.09714 18.85329 17.62742 25.19117
## [337] 17.19155 19.05780 23.83083 30.45211 17.87621 23.63281 29.43440 23.39822
## [345] 25.85293 19.80971 16.76977 21.77645 20.20183 14.14632
```

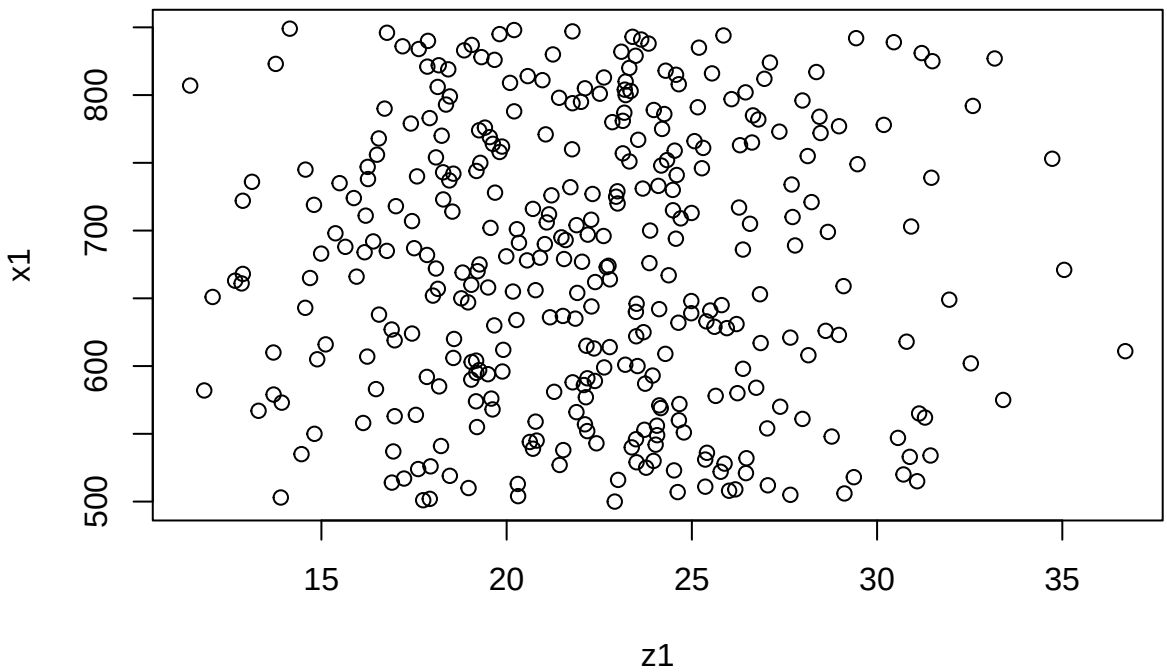
```r
w1<-length(z1)
w1
```

```
## [1] 350
```

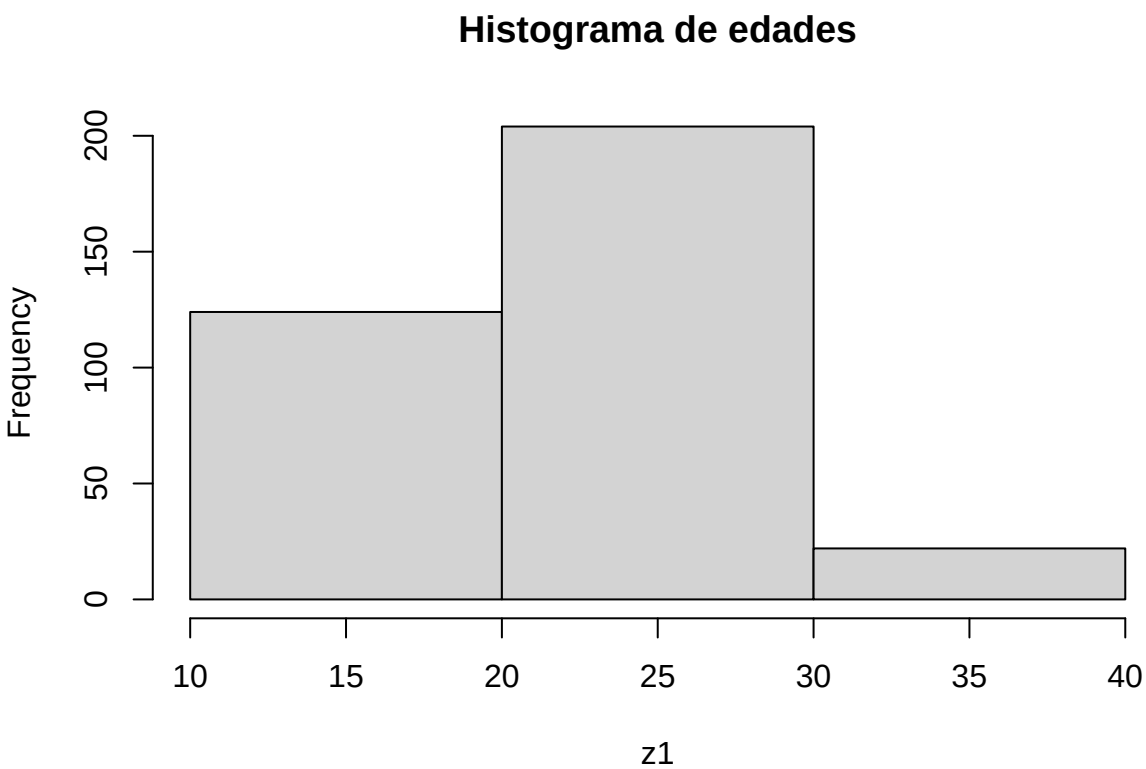
```r
x1<-(500:849)
x1
```

```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```r
plot(z1,x1)
```




```r
hist(z1,main="Histograma de edades",breaks=2)
```



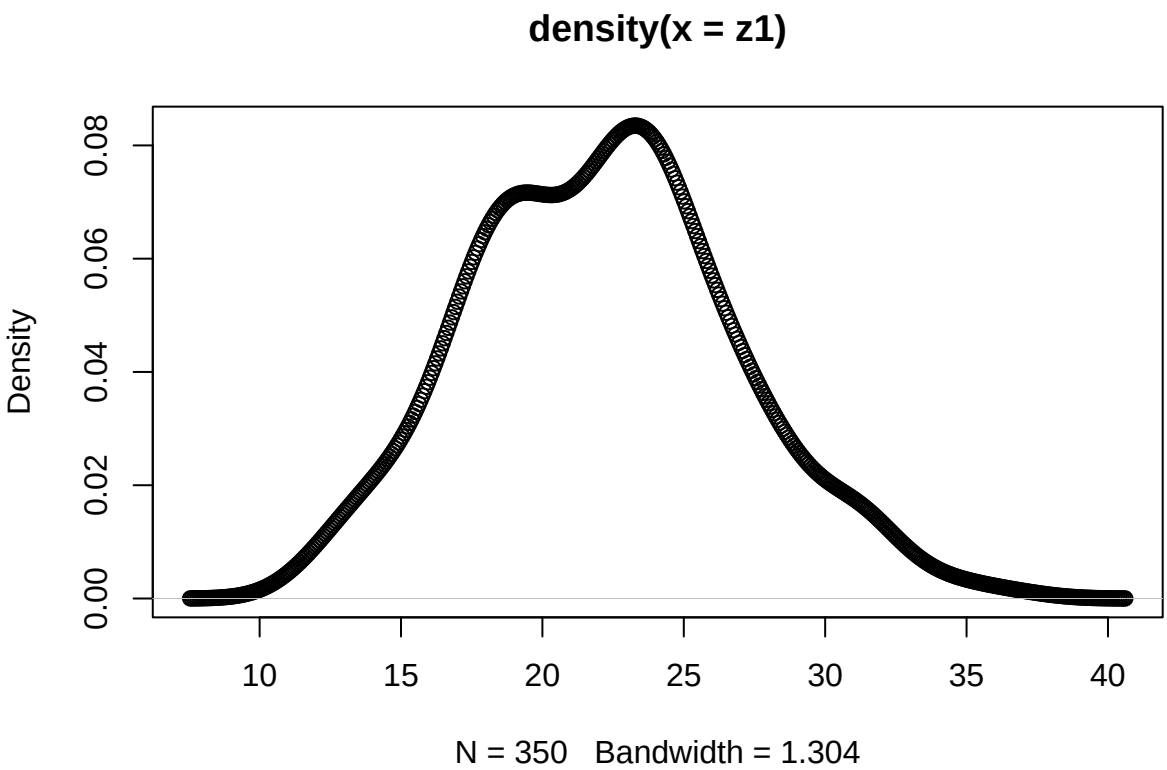

```r
density(z1,type='d')
```

```
## Warning: In density.default(z1, type = "d") :
##  extra argument 'type' will be disregarded
```

```
##
## Call:
```

```
##  density.default(x = z1, type = "d")
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.304
##
##        x                y
##  Min.   : 7.544   Min.   :9.907e-06
##  1st Qu.:15.812   1st Qu.:2.881e-03
##  Median :24.079   Median :1.980e-02
##  Mean   :24.079   Mean   :3.018e-02
##  3rd Qu.:32.346   3rd Qu.:5.947e-02
##  Max.   :40.613   Max.   :8.350e-02
```

```
plot(density(z1),type='b')
```




```
DNI=477
```

##Ejercicio 1 Consigna: Crear un vector que tenga todos los numeros enteros desde el 1 hasta DNI

```
vector_DNI=(1:DNI)
vector_DNI
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
```

```
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477
```

##Ejercicio 2 Calcular la suma de todos los valores del vector

```
Total=0
Valor_final=length(vector_DNI)
for (i in 1:Valor_final) {
  Total=Total+i}
  Total
```

```
## [1] 114003
```

##Ejercicio 3 Repetir el ejercicio anterior, pero en python

```
suma=0
DNI=477
for i in range(1, DNI):
  suma +=i
print(suma)
```

```
## 113526
```

##Ejercicio 4 ¿Cuanto tarda en correr el codigo del ejercicio 2?

```
Inicio=Sys.time()
Total=0
Valor_final=length(vector_DNI)
for (i in 1:Valor_final) {
  Total=Total+i}
  Total
```

```
## [1] 114003
```

```
Fin=Sys.time()
Fin-Inicio
```

```
## Time difference of 0.004939556 secs
```

##Ejercio 5 Calcular el ejercicio anterior usando la cabeza de gauss

```
n=length(vector_DNI)
suma=n*(n+1)/2
print(suma)
```

```
## [1] 114003
```

##CONSIGNA: Comparar la performance con systime

```
A <- 0
InicioA=Sys.time()
for (i in 1:50000) { A[i] <- (i*2)}
head (A)
```

```
## [1]  2  4  6  8 10 12
```

```
tail(A)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

```
FinA=Sys.time()
FinA-InicioA
```

```
## Time difference of 0.02552009 secs
```

```
InicioB=Sys.time()
B=seq(1, 100000, by = 2)
head(B)
```

```
## [1]  1  3  5  7  9 11
```

```
tail(B)
```

```
## [1] 99989 99991 99993 99995 99997 99999
```

```
FinB=Sys.time()
FinB-InicioB
```

```
## Time difference of 0.002599001 secs
```

##CONSIGNA: Cuantas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000 ?

```
a0=0
a1=1
it=2
  while (a1<1000001){
    an=a0+a1
    a0=a1
    a1=an
    it=it+1}
it
```

```
## [1] 32
```

##CONSIGNA: Compara la performance de ordenación del método burbuja vs el método sort de R.

```
library(microbenchmark)
# Tomo una muestra de 10 números ente 1 y 100
x<-sample(1:100,10)
# Creo una funci?n para ordenar
burbuja <- function(x){
n<-length(x)
for(j in 1:(n-1)){
for(i in 1:(n-j)){
if(x[i]>x[i+1]){
temp<-x[i]
x[i]<-x[i+1]
x[i+1]<-temp
}
```

```r
}
}
return(x)
}
res<-burbuja(x)
Bb<- microbenchmark(NULL, burbuja(x), times=1000L)
#Muestra obtenida
x
```

```
##  [1] 23 52 35 64 49 31 63 28 10 40
```

```r
#Muestra Ordenada
res
```

```
##  [1] 10 23 28 31 35 40 49 52 63 64
```

```r
#Ordanación con el coamando SORT de R-Cran
sort(x)
```

```
##  [1] 10 23 28 31 35 40 49 52 63 64
```

```r
Bs<- microbenchmark(NULL, sort(x), times=1000L)
#Benchmarks
Bb
```

```
## Unit: nanoseconds
##        expr  min   lq     mean median   uq   max neval
##        NULL    0    0    7.549     10   10    80  1000
##  burbuja(x) 6490 6880 7609.946   7060 7450 91491  1000
```

```r
Bs
```

```
## Unit: nanoseconds
##     expr   min    lq      mean median      uq    max neval
##     NULL     0     0    10.871     10    10.0    410  1000
##  sort(x) 22460 23780 27319.833  24480 25664.5 141890  1000
```